Задача 2

2a. Генерация строк с хвостовой рекурсией

Задан набор символов chars и целое число **n ≥ 0**.

Реализуйте функцию, которая возвращает последовательность всех строк длины n, составленных из символов chars, в которых никакие два соседних символа не совпадают.

```
(generate-strings chars n)

(generate-strings [\a \b \c] 2)
;; => ("ab" "ac" "ba" "bc" "ca" "cb")
;; с точностью до порядка элементов
```

Требования:

- использовать рекурсию и элементарные операции над последовательностями;
- все рекурсивные вызовы должны находиться в хвостовой позиции;
- для хвостовых рекурсивных переходов использовать recur;
- не использовать map, filter, reduce, for и другие средства, скрывающие обход последовательности;
- порядок строк в результате несущественен.

Для n = 0 считать результатом последовательность, содержащую пустую строку:

```
(generate-strings [\a \b] 0)
;; => ("")
```

2b. Та же задача с функциями высшего порядка

Решите задачу 2a повторно, используя стандартные функции высшего порядка для обработки последовательностей.

```
(generate-strings-fp chars n)

(generate-strings-fp [\a \b \c] 2)
;; => ("ab" "ac" "ba" "bc" "ca" "cb")
;; с точностью до порядка элементов
```

Требования:

- для обработки элементов коллекций использовать map, filter, reduce и/или другие стандартные операции над последовательностями;
- loop использовать запрещено
- явный рекурсивный обход коллекций запрещён;
- порядок строк в результате несущественен.

2c. Преобразование арифметического выражения в S-выражение

Дано арифметическое выражение в виде списка. Элементы списка чередуются: число, оператор, число, оператор, число и т. д.

```
'(1 + 4 * 3 - 5 / 2)
```

Допустимые операторы:

```
+  -  *  /
```

Реализуйте функцию:

```
(eval-expression expr)
```

Функция должна:

1. преобразовать исходный список в корректную форму Clojure;
2. учитывать приоритет операций: * и / имеют более высокий приоритет, чем + и -;
3. операции одного приоритета вычислять слева направо;
4. вычислить полученную форму с помощью eval.

Пример:

```
(eval-expression '(1 + 4 * 3 - 5 / 2))
;; => 21/2
```

Исходное выражение должно быть преобразовано в форму, эквивалентную:

```
(- (+ 1 (* 4 3))
   (/ 5 2))
```

Дополнительные примеры:

```
(eval-expression '(2 + 3 * 4))
;; => 14

(eval-expression '(10 - 3 - 2))
;; => 5

(eval-expression '(8 / 4 * 2))
;; => 4
```

Ограничение: итоговое арифметическое вычисление должно выполняться через eval. Не следует вычислять арифметические операции вручную через case, cond или аналогичный диспетчер операторов.